



PDF Download
3694715.3695969.pdf
29 March 2026
Total Citations: 9
Total Downloads: 2350



Published: 15 November 2024

Citation in BibTeX format

SOSP '24: ACM SIGOPS 30th Symposium
on Operating Systems Principles
November 4 - 6, 2024
TX, Austin, USA

Conference Sponsors:
SIGOPS

 Latest updates: <https://dl.acm.org/doi/10.1145/3694715.3695969>

RESEARCH-ARTICLE

Enabling Parallelism Hot Switching for Efficient Training of Large Language Models

HAO GE, Peking University, Beijing, China

FANGCHENG FU, Peking University, Beijing, China

HAOYANG LI, Peking University, Beijing, China

XUANYU WANG, Peking University, Beijing, China

SHENG LIN, Peking University, Beijing, China

YUJIE WANG, Peking University, Beijing, China

[View all](#)

Open Access Support provided by:

[Peking University](#)

[Purdue University](#)

Enabling Parallelism Hot Switching for Efficient Training of Large Language Models

Hao Ge^{*‡}
gehao@stu.pku.edu.cn
Peking University

Xuanyu Wang[‡]
wxyz0001@pku.edu.cn
Peking University

Xiaonan Nie[‡]
xiaonan.nie@pku.edu.cn
Peking University

Fangcheng Fu^{*†‡}
ccchengff@pku.edu.cn
Peking University

Sheng Lin[‡]
linsh@stu.pku.edu.cn
Peking University

Hailin Zhang[‡]
z.hl@pku.edu.cn
Peking University

Bin Cui^{†‡§}
bin.cui@pku.edu.cn
Peking University

Haoyang Li[‡]
lihaoyang@stu.pku.edu.cn
Peking University

Yujie Wang[‡]
alfredwang@pku.edu.cn
Peking University

Xupeng Miao
xupeng@purdue.edu
Purdue University

Abstract

Training of large-scale deep learning models necessitates parallelizing the model and data across numerous devices, and the choice of parallelism strategy substantially depends on the training workloads such as memory consumption, computation cost, and communication cost. Current approaches generally assume uniform training workloads across samples in a given task. Thus, existing systems are designed to adopt a static parallelism strategy throughout one training process. Nevertheless, when training models with sequence inputs, this assumption fails due to the sequence length variation across samples. Consequently, training with a static parallelism strategy would result in sub-optimal performance.

In this paper, we first reveal the under-explored fact that the optimal parallelism strategy varies even for the sequences within a single mini-batch. Motivated by this, we present

HotSPa, a novel system that adopts multiple parallelism strategies for efficient training with sequence inputs. To be specific, given a mini-batch of training sequences, HotSPa partitions them into multiple groups and applies different parallelism strategies to process each group individually. To enable the hot switching between strategies, HotSPa transfers model parameters and accumulated gradients among the devices on the fly. Significant solutions are proposed with the hope of seamless and rapid parallelism hot switching. Firstly, we design a graph compiler, which generates distributed computation graphs for different parallelism strategies simultaneously, and orchestrates them to share a single model storage backbone. Secondly, we develop a simple yet effective hot switch planner, which heuristically deduces communication plans to accelerate the transition of model partitioning given any pairs of strategies. Extensive experiments on large language model training demonstrate that HotSPa can be up to 2.99× faster than Megatron-LM and DeepSpeed that utilize static parallelism strategies. Source code is available: <https://github.com/PKU-DAIR/Hetu>.

CCS Concepts: • Computing methodologies → Distributed computing methodologies; Machine learning.

Keywords: Distributed Training, Large Language Model, Parallelism Strategy

ACM Reference Format:

Hao Ge, Fangcheng Fu, Haoyang Li, Xuanyu Wang, Sheng Lin, Yujie Wang, Xiaonan Nie, Hailin Zhang, Xupeng Miao, and Bin Cui. 2024. Enabling Parallelism Hot Switching for Efficient Training of Large Language Models. In *ACM SIGOPS 30th Symposium on Operating Systems Principles (SOSP '24)*, November 4–6, 2024, Austin, TX, USA. ACM, New York, NY, USA, 17 pages. <https://doi.org/10.1145/3694715.3695969>

^{*}Hao Ge and Fangcheng Fu contributed equally to this research.

[†]Fangcheng Fu and Bin Cui are the corresponding authors.

[‡]School of Computer Science & Key Lab of High Confidence Software Technologies (MOE), Peking University

[§]Institute of Computational Social Science, Peking University (Qingdao)

Permission to make digital or hard copies of all or part of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for components of this work owned by others than the author(s) must be honored. Abstracting with credit is permitted. To copy otherwise, or republish, to post on servers or to redistribute to lists, requires prior specific permission and/or a fee. Request permissions from permissions@acm.org. SOSP '24, November 4–6, 2024, Austin, TX, USA

© 2024 Copyright held by the owner/author(s). Publication rights licensed to ACM.

ACM ISBN 979-8-4007-1251-7/24/11

<https://doi.org/10.1145/3694715.3695969>

1 Introduction

Recent years have witnessed the revolutionary achievements of large Transformer models [69] in various fields like natural language processing [5, 16, 56], computer vision [17, 39, 41], and speech processing [2, 55, 70]. Among them, Large Language Models (LLMs), represented by ChatGPT and GPT-4 by OpenAI [51, 52], have garnered sweeping attention worldwide. The remarkable performance of LLMs is highly attributed to the increasing scale in model size and training data volumes [26]. In turn, this progressively makes the training of LLMs more challenging. Therefore, researchers and practitioners from both the academic and industry have aroused great interest in enhancing the training efficiency of LLMs.

Parallelism strategies such as model parallelism [20, 47, 48, 64] and sequence parallelism [21, 30, 35, 37, 40] have emerged as a cornerstone in addressing the shortage of device memory by distributing the model states and/or intermediate results in model computation across devices. Although parallelism strategies offer a significant venue in supporting large-scale models, they also come with certain trade-offs. In particular, they introduce communication overhead to exchange information among different devices, hampering the training efficiency. When facing scenarios that demand higher memory requirements, such as larger model size and/or longer context length (i.e., the maximum supported sequence length of the model), we usually have to increase the parallelism degrees to leverage a more memory-saving parallelism strategy, at the price of enlarged communication overhead simultaneously. Existing LLM training systems, such as Megatron-LM [49, 64] and DeepSpeed [59], usually encompass many (if not all) parallelism strategies, and allow users to strike a balance between memory consumption and training efficiency by configuring a suitable parallelism strategy.

(Motivation) In practice, these systems leverage a *static parallelism strategy* for a single training task. To be specific, after a parallelism strategy has been configured, it will be fixed throughout the training task. The reason is that they ideally assume the training workloads (i.e., computation cost, communication cost, and memory consumption) are static across different samples (i.e., sequences). Nevertheless, although such an assumption is reasonable for tabular datasets, it is not true for Transformer models.

In a nutshell, LLMs target at sequence-to-sequence tasks, which generate new sequences given input sequences. During training, after we have randomly drawn a mini-batch of sequences, their lengths inevitably vary. As we will discuss in more details in Section 3, the lengths of sequences in widely used datasets typically follow an extremely skewed distribution — most of the sequences (e.g., over 86%) have lengths shorter than 1024 whilst just a handful of them (e.g., less than 0.5%) are longer than 8192. Undoubtedly, such a phenomenon of *sequence length variation* results in imbalanced workloads across different sequences.

The mismatch between the unrealistic assumption in existing systems and the workload imbalance in training sequences poses a significant challenge when short and long sequences are processed together. Specifically, it is well known that the training of long sequences entails much more memory consumption than that of short sequences, as a result of which, existing systems are forced to configure the parallelism strategy to be more memory-saving in order to avoid out-of-memory errors. Nevertheless, although the configuration sounds reasonable for long sequences, it would prolong the processing time of short sequences — as discussed above, memory-saving parallelism strategies usually entail more communication overhead, yet short sequences could be accommodated by some less memory-saving but more efficient strategies. Worse still, considering the fact that increasing the context length is also one of the most intriguing trends in the field of LLMs, the challenge would be even severer.

In short, previous approaches ignore the workload imbalance in LLM training and simply choose a static parallelism strategy. Thus, we try to accelerate LLM training from an unexplored aspect: *can we adopt separate parallelism strategies for sequences with different levels of workloads/lengths?*

(Summary of Contributions) To achieve this goal, we develop HotSPa, a brand new system for efficient LLM training. The major contributions are summarized below.

C1: Proposal of Parallelism Hot Switching. We first conduct a small testbed to show that during LLM training, the processing of most sequences is far from optimized due to the skewness in training samples. Based on the findings, we propose a novel training paradigm with parallelism hot switching in HotSPa. Given a mini-batch of training samples, HotSPa classifies them into several groups according to the workload differences, and trains each group with the most suitable parallelism strategy. The gradients of different groups are accumulated before model update to ensure the model convergence is not affected. When two groups are configured with different parallelism strategies, HotSPa automatically transfers model parameters and gradients on the fly to seamlessly switch from one strategy to another.

C2: Unified Representation and Graph Compilation. Current systems (e.g., Megatron-LM and DeepSpeed) have significantly different implementations for different parallelism strategies, which hinders the possibility of using separate strategies for sequences with different workloads. To this end, our work designs a graph compiler for the compilation and deduction of distributed computation graphs to support parallelism hot switching in LLM training. To be specific, HotSPa provides a unified abstraction and implementation to demonstrate multiple parallelism strategies in one single logical graph. Then, the graph compiler generates multiple executable computation graphs that share the same backbone storage of model states. In particular, based on analysis regarding storage size and communication cost, the

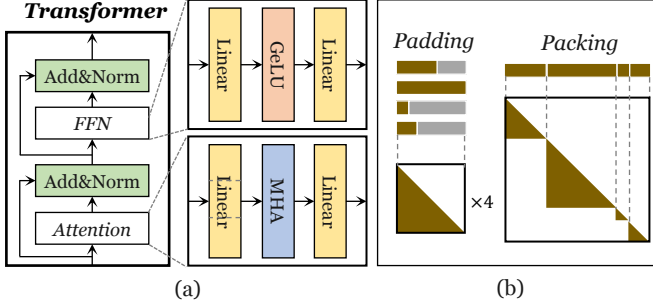


Figure 1. Illustration of (a) the architecture of Transformer layer; (b) sequence padding and packing.

graph compiler selects only one graph to instantiate the storage of model states, whilst the other graphs are temporary and require model hot switching.

C3: Hot Switching Optimization. Given the candidate parallelism strategies, in order to efficiently switch from one strategy to another, it is necessary to exchange the model parameters and accumulated gradients among the devices, resulting in substantial communication overhead given the huge scale of LLMs. To address this problem, we devise a heuristic algorithm that deduces efficient many-to-many communication plans of parallelism hot switching. In particular, the heuristic planner takes the characteristics of modern network connections into account, and greedily balances both the intra- and inter-node communication loads. Furthermore, a layout optimization technique is developed to get rid of non-sequential memory accesses in hot switching.

C4: Implementation and Evaluation. We build the HotSPa system, enabling the brand new LLM training paradigm of parallelism hot switching. Comprehensive experiments demonstrate that HotSPa achieves up to 2.99 $\times$ of speedup compared to state-of-the-art LLM training systems for different sizes of LLaMA2-series LLMs and different levels of context lengths. Additionally, we conduct case studies to anatomize the performance gains of HotSPa with more details, showcasing the power of parallelism hot switching.

2 Preliminaries

In this section, we briefly walk through the preliminary literature related to our work. Throughout this paper, the term “context length” (denoted by s) refers to the maximal number of tokens that a model supports, and “sequence length” refers to the number of tokens of some specific training sequence.

2.1 Transformers and Large Language Models

Transformer [69] is first proposed for sequence-to-sequence modeling tasks such as machine translation and language understanding, and has become the foundational architecture of most LLMs. There are two basic blocks in a Transformer layer, which are the self-attention network and point-wise feed-forward network (FFN), as depicted in Figure 1(a).

Padding and Packing. Unlike tabular datasets where all samples are well-structured, sequences are of varying

lengths in nature. In order to train sequences with different lengths together, techniques such as padding or packing are needed to preprocess the sequences, as illustrated in Figure 1(b). Padding is a widely used technique, which pads or truncates the sequences in the same mini-batch to be of the same length. However, padding results in the waste of computation, communication, and memory, especially when the sequence lengths are highly skewed. Sequence packing [31] is a more advanced technique to cope with the wastes due to sequence length variation. With this technique, multiple sequences are packed into a single one. As shown in Figure 1(b), to prevent cross-contamination among these sequences, adjustments to the attention mask and position indexing are needed so that the sequences do not attend to each other. State-of-the-art implementations of the attention mechanism (e.g., FlashAttention [12, 13]) have provided functionality for the attention computation on a packed sequence.

Since packing is usually more efficient than padding, we focus on packing in the rest of this paper. However, it is worthy to note that the idea of parallelism hot switching is compatible with both techniques.

2.2 Deep Learning Training

Computation Graph. Deep neural networks are usually depicted via *computational graphs*, where each node represents an operation and each edge represents a tensor or execution dependency [1]. Model training involves iterative forward and backward propagation passes. During the forward propagation, a mini-batch of training samples is fed into the computation graph, computing the outputs (a.k.a. forward activations) of all layers. The final outputs are compared with the expected values via a pre-defined loss function. During the backward propagation, the errors are propagated to compute the activation gradients (a.k.a. backward derivatives) and model gradients. The model parameters will be updated with the model gradients by a pre-defined optimizer.

Mixed-precision Training. With the rapid growth of model size, it has become a de facto choice to leverage the mixed-precision technique [46] when training large-scale models. To be specific, before the forward propagation, model parameters are first casted from 32-bit floating points (i.e., FP32) to 16-bit floating points (i.e., FP16 or BF16). Then, all the values computed during the training process, including forward activations, backward derivatives, and model gradients, can be represented in the 16-bit format. By doing so, the memory, computation, and communication requirements can be reduced simultaneously. However, to preserve the model convergence, model parameters and optimizer states (e.g., the first- and second-order momentums in Adam optimizer [28, 42]) are still maintained in the 32-bit format.

Gradient Accumulation. To stabilize and improve the convergence, modern LLMs are usually trained with large batches (typically, a few millions of tokens in one batch [11,

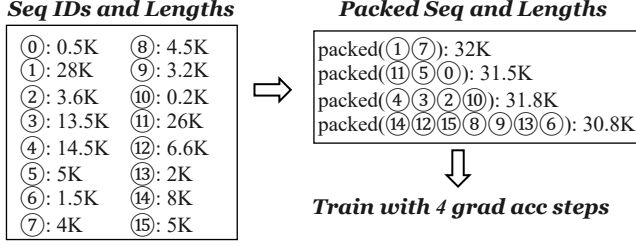


Figure 2. An example of training with gradient accumulation and sequence packing when the maximal supported length is 32K.

19, 67], which corresponds to thousands of sequences in real-world datasets). As memory consumption of activations is proportional to the number of tokens, the hardware may not have sufficient memory to process a large batch of sequences at once. Gradient accumulation is such a technique that slices a large batch into $p > 1$ micro-batches, computes the model gradients with these micro-batches individually, and accumulates the gradients. The model parameters will be updated with the accumulated gradients of p micro-batches.

When working with sequence packing, each micro-batch represents a packed sequence. As shown in Figure 2, given a batch of sequences and the maximal supported length (i.e., the maximal supported number of tokens to process at once without encountering out-of-memory errors), we can produce multiple packed sequences. Then, these packed sequences are processed individually and their corresponding model gradients are accumulated for model update.

2.3 Parallelism Strategies

Below we introduce widely used parallelism strategies for LLM training.

Data Parallelism. In data parallelism (DP) [15, 38, 63], training samples are partitioned across devices, and each device maintains a full copy of model states (i.e., model parameters and optimizer states). In each iteration, all devices execute the forward and backward propagation locally, and synchronize the model gradients globally (e.g., by all-reduce). However, as the model size increases, maintaining a full copy of model states on each device becomes infeasible. Thus, Rajbhandari et al. [57] proposed the Zero Redundancy Optimizer (ZeRO), which evenly partitions model states across devices. There are three stages of ZeRO, namely ZeRO-1, -2, and -3, respectively. Among them, ZeRO-3 (a.k.a. FSDP [25, 79]) partitions all model states and has the lowest memory consumption. However, an extra round of all-gather communication of model parameters is needed for each batch. Worse still, when working with gradient accumulation, the extra all-gather needs to be executed for each micro-batch. Thus, when there are p micro-batches, the communication cost of ZeRO-3 would be $p + 1/2$ times of that of conventional DP.

Model Parallelism. In model parallelism, a model is split across devices, and the forward and backward propagation

must be done in a distributed manner. There are two major types of model parallelism, namely tensor parallelism (TP) [64] and pipeline parallelism (PP) [20, 47, 48]. In TP, each model parameter is partitioned, and the intermediate results (forward activations and/or backward derivatives) on each device would be incomplete/partial, so communication operations (e.g., by all-reduce) are needed. The most widely used TP strategy is proposed by Megatron-LM [64], where two consecutive parameters within one self-attention or FFN block are split in the column- and row-wise, respectively. Eventually, there will be 4 all-reduce communication for one Transformer layer. In PP, a model is reckoned as a sequence of layers, and divided into multiple stages across devices. P2P communication operations are needed to transfer the intermediate results between any two consecutive stages.

Sequence Parallelism. Sequence parallelism (SP) [21, 30, 35, 37, 40] can be deemed as a special form of DP. SP splits the training samples (sequences) in the sequence dimension, which is helpful in amortizing the memory of forward activations, especially when handling long sequences. According to the modules in which sequence partitioning is employed, there are different types of SP, and their memory consumption and communication complexities vary. For instance, The Megatron-style SP [30] is applied in the dropout and normalization modules, but demands extra all-gather and reduce-scatter communication operations. In addition, it requires that SP must be used alongside TP, and the parallelism degrees of SP and TP must equal. As another notable approach, DeepSpeed employs the Ulysses-style SP [21] in dropout, normalization, and the linear projections in the self-attention block, whilst 4 rounds of all-to-all communication would be needed to re-partition the sequences. However, it necessitates complete model parameters of the linear projections on each device, making it incompatible with TP.

Hybrid Parallelism. Since different parallelism strategies exhibit unique performance in terms of computation, communication, and memory, modern training systems combine multiple strategies into a hybrid parallelism strategy for better efficiency. Notably, Megatron-LM supports the 3D parallelism strategy [49], which encompasses DP, TP, and PP. In particular, engineers or researchers can tune the parallelism degree of each strategy for a given task to achieve better efficiency. To ease the laborious tuning efforts, there are studies exploring automatic parallelism [23, 44, 66, 74, 81], which automatically deduces a suitable (hybrid) parallelism strategy given a training task. Nevertheless, all these works assume the workloads across samples (in terms of memory consumption and running time) are fixed, so they leverage a static parallelism strategy throughout the training task.

3 Observations and Opportunities

Before stepping into the design of HotSPa, we would like to first introduce two observations that motivate our work.

Table 1. Running time (in seconds) of different tensor parallelism degrees (LLaMA2-7B, 8GPUs, $DP=8/TP$) when processing the same amount of tokens.

Seq Len	# Seqs	$TP=1$	$TP=2$	$TP=4$	$TP=8$
1K	512	13.9	14.7	16.2	19.5
2K	256	14.2	15.1	16.6	19.8
4K	128	14.9	15.8	17.4	20.6
8K	64	OOM	17.4	19.1	22.1
16K	32	OOM	OOM	21.8	25.0
32K	16	OOM	OOM	OOM	30.8

Observation 1: Efficiency Degradation for Longer Context Lengths. Training LLMs with longer sequences inevitably increases memory consumption greatly. Therefore, when extending the context length of LLMs, it is common knowledge that we have to trade time for memory. In other words, it is necessary to increase the model and/or sequence parallelism degrees to accommodate the memory consumption, yet at the price of higher communication overhead.

We choose TP as a representative and conduct experiments to examine the efficiency degradation. As shown in Table 1, when handling relatively short sequences, parallelism strategies with higher DP degrees always achieve better efficiency since they incur less communication overhead. However, these strategies gradually run out of memory as the context length increases, which is reasonable since the memory footprints grow accordingly. Consequently, we need to adopt parallelism strategies that are more memory-saving, even though they are less efficient. In short, when processing the same amount of tokens, the parallelism strategies differ in terms of efficiency and memory consumption, yet the context length is a crucial factor in determining whether an efficient strategy is able to accommodate the memory consumption.

Observation 2: Skewness in Sequence Lengths. Figure 3 shows the distributions of sequence lengths in two popular datasets for LLMs, which are *CommonCrawl* and *GitHub*. We observe significant skewness in the sequence lengths. For instance, in the *CommonCrawl* dataset, one of the most widely used corpus for LLM training, over 86% of the sequences have fewer than 1K tokens, whilst sequences longer than 8K are less than 0.5%. Although the *GitHub* dataset contains more long sequences as codes can be much longer than human texts, the skewness is still high. In fact, such a skewness phenomenon is common in human texts and has also been observed in diverse datasets for LLM training (e.g., Section 2 of [14]). As a consequence, during training, when we randomly sample a mini-batch of sequences, only a handful of them are with long lengths whilst most of them are relatively short. Recall that in order to accommodate the memory consumption for long sequences, more memory-saving parallelism strategies would be adopted, rendering

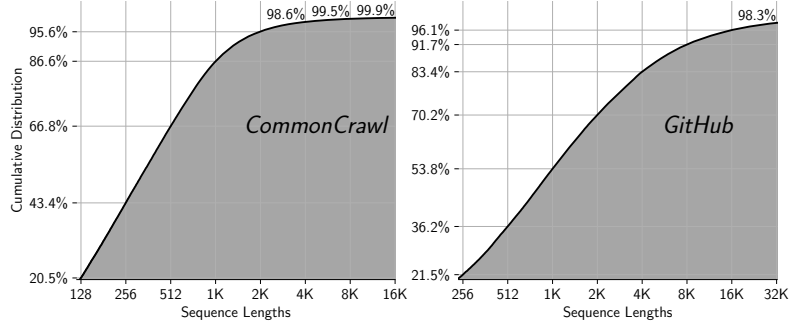


Figure 3. Cumulative distributions of sequence lengths.

around 40% of efficiency degradation for short sequences if we consider the results in Table 1.

Opportunity and Rationality. Although Table 1 only takes TP as a representative and Figure 3 only plots the distributions of two datasets, they still reveal a universal fact that during the training of LLMs, short sequences are incapable of being processed with efficient strategies due to the existence of long sequences, making the majority of training samples not fully accelerated.

Motivated as such, we try to harness this limitation and accelerate LLM training from a different perspective that *sequences with different levels of lengths should be processed with different parallelism strategies*. For instance with Table 1 again, we can classify the sequences in a mini-batch into two groups, where the first group consists of sequences with less than 4K tokens and the second group consists of the others. For the second group (long sequences), we must adopt a memory-saving strategy to process them to avoid out-of-memory errors. However, for the first group (short sequences), we can switch to the most efficient strategy (i.e., the $DP=8$ strategy¹). By doing so, we can speed up the processing of more than 98% and 83% of the sequences in the two datasets, respectively.

Training with such a parallelism hot switching paradigm necessitates computing model gradients for different strategies separately, and incurs the communication overhead for re-partitioning the model on the fly. Nevertheless, we find that it suits LLM training well. For one thing, modern LLM architectures do not involve batch-sensitive modules (e.g. batch normalization), making the gradients of different samples accumulable. (In fact, gradient accumulation has already been an essential technique in LLM training.) Thus, as long as the model gradients computed by different strategies are accumulated together, it achieves identical model updates as training with a static parallelism strategy. For another, as introduced in Section 2, modern LLMs are usually trained with large batches, so the overhead brought by model re-partitioning can be amortized greatly since it does not increase proportionally to the batch size.

¹The sequences in the first group are packed with a maximal length of 4K to avoid out-of-memory errors when processing with $DP=8$.

4 HotSPa

4.1 Overview

We present the overall routine of HotSPa in Figure 4. There are two major steps in total. The first major step is to analyze and generate the execution plans. Given the model and the candidate parallelism strategies, HotSPa uses a heuristic algorithm to deduce the many-to-many communication plans, compiles the distributed computation graphs for all strategies, and orchestrates them to enable training with parallelism hot switching. A graph will be proposed for instantiation (i.e., HotSPa will allocate and initialize model parameters and optimizer states via this graph). We will introduce these components in Section 4.2 and Section 4.3.

After the computation graphs and hot switching plans are ready, and model states have been instantiated, HotSPa forward to the second major step, which is the iterative training process. Given a mini-batch of sequences, they are divided into different groups according to the strategies (①). For each group, HotSPa transmits the model parameters and/or accumulated gradients to the desired strategy, runs the computation graph, and accumulates the gradients (②-⑧). At the end of each mini-batch, the model states are updated using the instantiation graph (⑨). It is worth noting that HotSPa accumulates the gradients computed by different strategies, so the model update is identical to that of training with a single strategy, and does not affect the convergence.

4.2 Graph Compilation

In order to support the seamless transition between any pair of parallelism strategies, it is essential to have a unified representation that denotes and tracks each parameter’s device mappings for candidate parallelism strategies. Offering such an abstraction not only eases the implementation of training with multiple strategies but also facilitates the compilation and optimization of distributed executions.

Definition of Logical Graph. To begin with, HotSPa uses a unified representation, namely the *DistConfig*, to specify the distributed configuration (i.e., partitioning and placement) of a model parameter under a parallelism strategy. As shown in Figure 5, a *DistConfig* consists of three major members: the assigned devices, the number of duplications to indicate DP degree, and a map to indicate how a multi-dimensional parameter is split among the devices, which is essential for TP. Besides, PP is automatically identified if two consecutive parameters are located on different devices.

Based on the *DistConfig* abstraction, HotSPa defines multiple parallelism strategies in one *LogicGraph* efficiently. In particular, *LogicGraph* excludes all communication, but only defines the computation logics of the model, and each parameter is associated with multiple *DistConfigs*. As shown in Figure 5, each set of *DistConfigs* represents one parallelism strategy. By doing so, we can get rid of the redundant definitions for different strategies one by one, offering convenient APIs to define and analyze the model training.

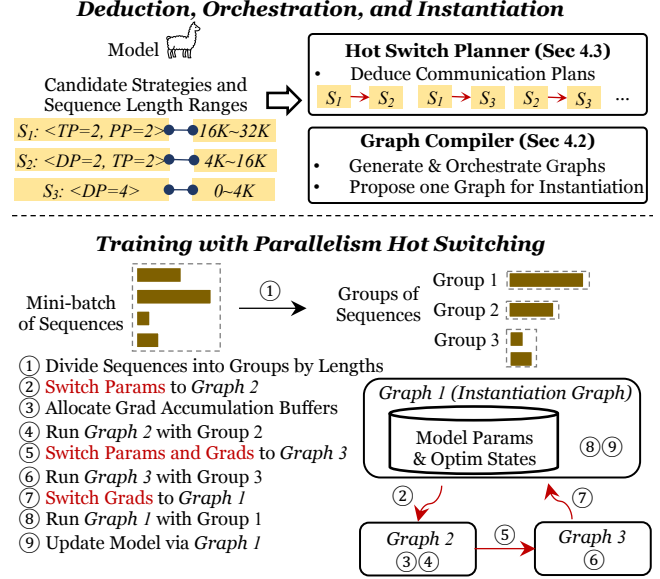


Figure 4. The overall routine of HotSPa, which takes as input a model, multiple candidate parallelism strategies, and a mapping from each strategy to the range of sequence lengths it supports. We mark the parallelism hot switching during training in red.

The *DistConfig* abstraction in our work is simple yet general to describe popular parallelism strategies in LLM training. It is worthy to note that although similar tensor annotations for distributed computation graphs have been proposed in prior works like GSPMD [76] and Unity [68], our work differs from them in goals — these prior works do not support annotating a tensor with multiple strategies simultaneously, nor the orchestration of different candidate strategies.

Given the *LogicGraph* of a training task, along with multiple sets of *DistConfigs* representing the candidate parallelism strategies, there are two phases of compilation to support training with parallelism hot switching, as introduced below.

Phase 1: Generation of Executable Graphs. The *LogicGraph* will first be compiled into multiple *ExecGraphs*, where each *ExecGraph* stands for the executable distributed computation graph of one strategy. Specifically, as the *LogicGraph* merely defines the computation logics, the *ExecGraphs* are generated considering three types of insertions.

- For model parameters, the type casting operators are inserted, which cast values from 32-bit into 16-bit floating points, in order to support mixed-precision training.
- Inside the forward and backward propagation, communication operators, such as all-reduce operators for TP and P2P send/rcv operators for PP, are inserted if needed.
- For model gradients, accumulation operators are inserted for the sake of memory. Communication operators will also be inserted for gradient synchronization if the corresponding parameters are marked as duplicated (i.e., DP).

In fact, there are many other important compilation passes to enhance efficiency, such as operator fusion, in-place memory

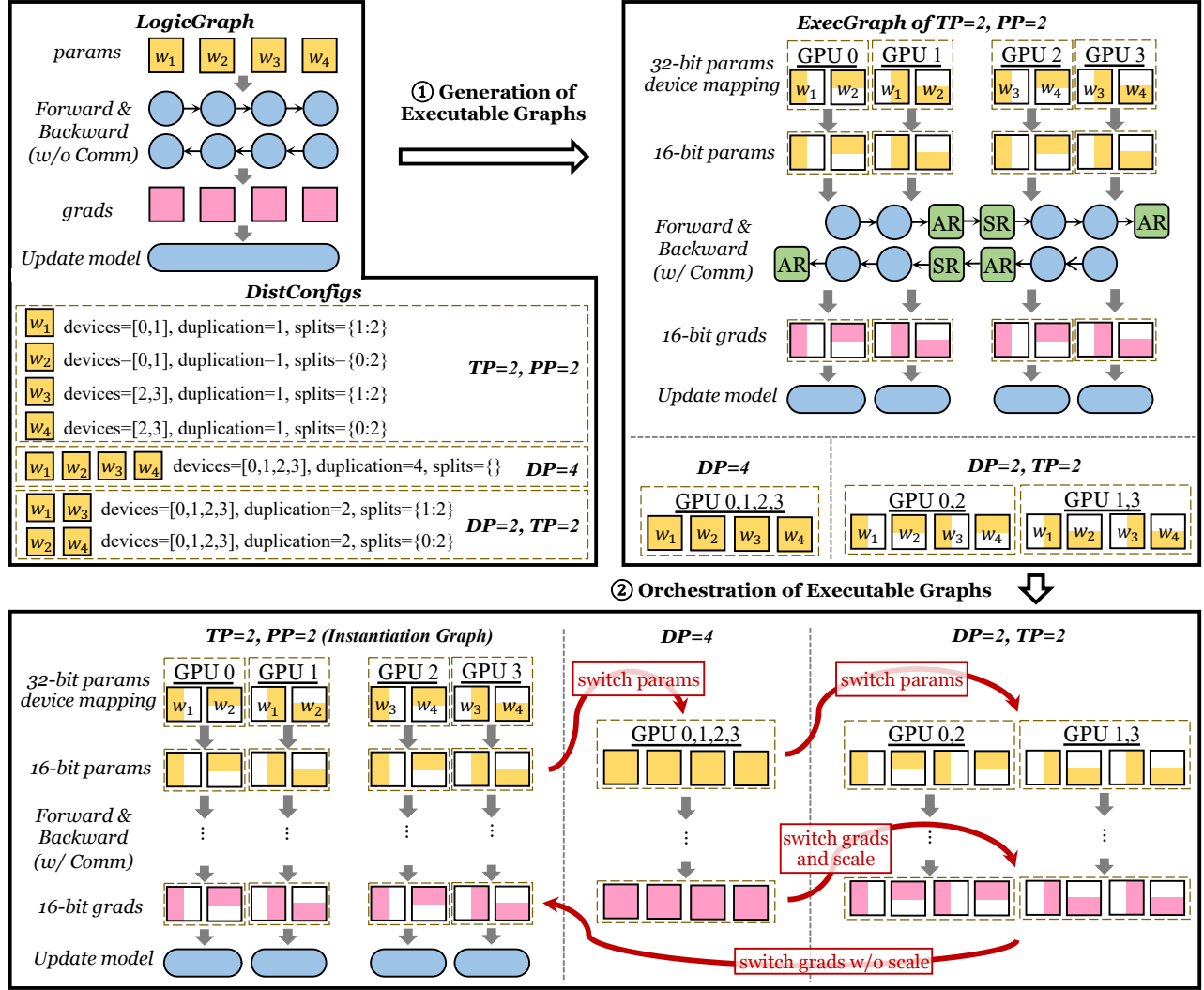


Figure 5. Illustration of the graph compilation process. We omit the gradient accumulation operations for simplicity. “AR” and “SR” represent All-Reduce and P2P Send/Recv communication, respectively.

sharing, and topology scheduling [9, 22, 80]. In this paper, we only introduce the insertions that are demanded to make *ExecGraphs* self-contained and suitable for parallelism hot switching. Interested readers are referred to related studies for more details about graph compilation [36].

Phase 2: Orchestration of Executable Graphs. Given the astonishing scale of LLMs, it is infeasible to allocate model states for all parallelism strategies. Thus, after the *ExecGraphs* are generated, we then orchestrate them together to ensure only one copy of model states will be instantiated, and to reduce the communication overhead due to the transition among strategies. There are three steps in total.

Firstly, we need to identify the candidate *ExecGraphs* that are suitable for instantiation. In particular, we wish to minimize the memory occupation of model states, which is essential for the successful execution of all strategies — imagine

that if model states are allocated and maintained according to a memory-intensive strategy (e.g., full DP), then it is very likely that memory-saving strategies would run out of memory since model states take more space than expected. Considering that there may be two or more graphs satisfying our requirement (e.g., both $\langle TP=4 \rangle$ and $\langle TP=2, PP=2 \rangle$ minimize the memory occupation of model states when $N = 4$), we simply mark them as candidate graphs for instantiation, and the final decision will be made in the next step.

Secondly, the *ExecGraphs* are re-ordered for better efficiency. To be specific, the hot switching cost between any pair of *ExecGraphs* is analyzed (detailed in Section 4.3). Then, we will enumerate all feasible ordering of *ExecGraphs*, examine their total hot switching cost, and select the best one. (The ordering must start from the candidate graphs for instantiation as discussed above). And the first graph is chosen for instantiation. For instance, in Figure 5, the graphs are

re-ordered as $\langle TP=2, PP=2 \rangle$, $\langle DP=4 \rangle$, and $\langle DP=2, TP=2 \rangle$. By doing so, only the first parameter switching (i.e., from $\langle TP=2, PP=2 \rangle$ to $\langle DP=4 \rangle$) necessitates cross-device communications, whilst in the second parameter switching and both times of gradient switching, only standalone memory copy and slicing are needed to adjust the model partitioning, incurring zero network overheads.

Thirdly, since model states are allocated and maintained according to the instantiation *ExecGraph*, we prune the type casting and model update operators of the other *ExecGraphs*. Subsequently, all *ExecGraphs* are chained with corresponding hot switching plans. Moreover, model parameters are first cast to the 16-bit version before the first hot switching in order to halve the communication cost. Eventually, only 16-bit values (parameters and gradients) are transferred across parallelism strategies.

Harnessing the DP Degrees. When a model parameter is marked as duplicated (i.e., with a DP degree greater than 1), each device would only own an in-completed (a.k.a. partial) copy of the corresponding model gradient after backward computation and gradient accumulation. When training with static parallelism strategies, it is necessary to turn the model gradient from partial to completed (i.e., duplicated) via communication. Whilst in HotSPa, when the source and destination parallelism strategies have the same DP degrees, we further enhance overall efficiency by pruning the gradient synchronization in the source *ExecGraph*.

However, when the source and destination strategies have different DP degrees, the gradient synchronization in the source *ExecGraph* remains necessary. In addition, after a model gradient is switched, we shall divide the gradient by the corresponding DP degree to make it partial. By doing so, we can directly re-use the memory of the partial gradient as the accumulation buffer, and meanwhile guarantee the correctness of hot switching training.

Discussion. HotSPa only maintains one copy of 32-bit model states globally according to the instantiation *ExecGraph*, whilst the other *ExecGraphs* share the same model storage via hot switching. As aforementioned, when determining the instantiation *ExecGraph*, only those with minimal memory occupation in terms of model states would be considered. Therefore, during the execution of any *ExecGraph*, the memory consumption is at least the same as training with static parallelism strategies. This ensures HotSPa accommodates the memory requirements of all strategies.

Furthermore, same as the 32-bit model parameters, the optimizer states are maintained according to the instantiation *ExecGraph*. During training, only the 16-bit model parameters and accumulated gradients are switched among parallelism strategies. Therefore, the size of optimizer states does not impact the hot switching cost, which also indicates our work can be integrated with various kinds of optimizers.

Algorithm 1: Our heuristic hot switching planner.

```

1 Initialize hot switching plan  $\mathcal{P} = \{\}$ ;
2 Initialize intra- and inter-node communication
   volume  $V_i^{(inter)}, V_i^{(intra)}$  as 0 for each device  $i$ ;
3 foreach model parameter/gradient slice do
4   Determine the owner (source) devices  $S$ ;
5   Determine the target (destination) devices  $D$ ;
6   foreach  $dst$  in  $D$  do
7     if  $dst \notin S$  then
8       Partition  $S$  into  $S^{(intra)}, S^{(inter)}$ ;
9       if  $S^{(intra)}$  is not empty then
10         $src \leftarrow \arg \min_i \{V_i^{(intra)} | i \in S^{(intra)}\}$ ;
11         $V_{src}^{(intra)} \leftarrow V_{src}^{(intra)} + \text{sizeof}(slice)$ ;
12      else
13         $src \leftarrow \arg \min_i \{V_i^{(inter)} | i \in S^{(inter)}\}$ ;
14         $V_{src}^{(inter)} \leftarrow V_{src}^{(inter)} + \text{sizeof}(slice)$ ;
15       $\mathcal{P} \leftarrow \mathcal{P} \cup (slice, src, dst)$ ;
16 return  $\mathcal{P}$ ;

```

4.3 Model Hot Switching

Unlike regular collective communication in DNN training (e.g., broadcast, reduce, etc.), the hot switching process involves the re-partitioning of model parameters or gradients among the devices, which is a many-to-many communication. Given that each model parameter may have different numbers of source devices and destination devices (depending on the DP degrees), there exist numerous possible communication plans. However, how to determine the optimal plan is extremely challenging. For one thing, since each model may be split into different numbers of slices in the source and destination strategies, and devices in the destination strategy may look for different slices, there will be a vast number of possible plans. For another, if two P2P communication paths share the same network switch, they inevitably interfere with each other, making it complicated to theoretically analyze the time cost of each P2P communication.

Although some prior works have designed the resharding of activations [81, 82], they cannot be applied to our work. Firstly, they focus on cross-mesh resharding, where the source mesh and destination mesh consist of different, non-overlapping, sets of devices, whilst we should also consider in-place memory copy and intra-node communication to repartition the model. Secondly, the prior works aim to minimize the communication of each activation individually since different activations are computed at different time, whilst our work should consider the problem from a global perspective to fuse and minimize the communication of multiple layers. Thus, the communication plans to consider in our work are more diverse and complex.

Heuristic Solution. Despite the difficulty discussed above, we can take key characteristics of networks in GPU clusters into account when deducing a suitable hot switching plan. Below we consider two characteristics, and introduce the resulting heuristics, respectively.

- *Intra-node communication is preferable than inter-node:* In typical GPU clusters, GPUs within a node are connected by NVLink, which has a higher communication bandwidth than Infini-band or Ethernet, which are usually utilized to connect GPUs across nodes. Therefore, if there are two or more sources for a model slice, we can prioritize intra-node communication whenever possible.
- *GPU connectivities are full-duplex:* Modern network connections typically have separate bandwidth for sending and receiving. For each device, data transmissions in both directions simultaneously do not interfere with each other. Given the fact that the total amount of needed slices for each device is specified by the destination parallelism strategy, when deducing the hot switching plan, it is reasonable to minimize the maximum sending volume of all devices.

Based on these two heuristics, we design a simple yet effective greedy algorithm for the planning of model hot switching, as shown in Algorithm 1. Briefly speaking, we record the intra- and inter-node communication volumes for each device, respectively (line 2). For each model slice, we first determine the owner and target devices according to the *DistConfigs* (lines 4-5). Based on the first heuristic, for each target device, we examine whether there are intra-node owner devices to make full use of the faster intra-node bandwidth (lines 8-14). Furthermore, based on the second heuristic, among the intra- or inter-node candidates, we greedily choose the one with minimal sending volume so far (lines 10-11 and 13-14). Additionally, the maximum inter-node communication volume will be used to describe the price of hot switching for the re-ordering of *ExecGraphs* in Section 4.2.

Although Algorithm 1 may not achieve the optimal plan, it suits real-world GPU clusters well and achieves a good load balance in terms of the communication volume among all devices. As we will show in Section 5, the hot switching cost is worthy given the improvement in training efficiency.

Layout Optimization. As introduced in Section 2, there are two widely used parallel partitioning approaches for TP, which are column- and row-partitioning, respectively. In our scenario of model hot switching, when the parallelism degree of column-partitioning changes, extra concatenation and copy operations are needed to adjust the memory layout accordingly, as depicted in Figure 6. These layout changes must be done via non-sequential memory accesses, which are not friendly for parallel processing accelerators like GPUs.

To cope with this problem, we leverage a layout optimization technique to further enhance the efficiency of model hot switching. To be specific, for all column-partitioned parameters, we maintain them as column-major ordered (i.e.,

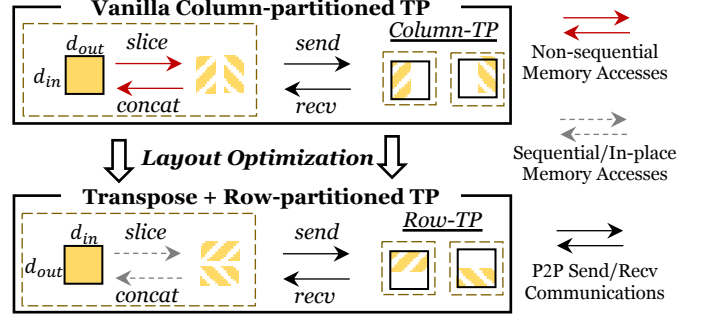


Figure 6. Layout optimization for column-partitioned TP. We transpose the models and leverage row-partitioned TP to avoid non-sequential memory accesses when increasing (from left to right) or decreasing (from right to left) the TP degree.

in the transposed view) rather than row-major ordered. During the training, we can simply flip the layout argument when launching the gemm kernel in libraries such as cuBLAS or cutlass. By such means, we can avoid non-sequential memory accesses, whilst not affecting the training routine.

4.4 Implementation

Although current systems have supported various parallelism strategies and degrees, they can only designate a single, static parallelism strategy for a single training task. If we wish to switch the parallelism strategy with current systems, we must store the latest model states and accumulated gradients into a checkpoint, re-start the training task with a new strategy, and load the checkpoint to continue the training of the current mini-batch. Undoubtedly, this is impractical.

To support training with parallelism hot switching, HotSPA is built on top of Hetu² [43], our self-developed DL system consisting of 55K LoC in C/C++/CUDA and 2.4K LoC in Python. Hetu implements the *DistConfigs* as a set of annotations to describe the sharding, duplication (a.k.a. replication), and partial statuses of tensors. To define multiple parallelism strategies, Hetu provides APIs to annotate the tensors in the *LogicGraph* with several sets of annotations, where each set of annotations expresses one strategy. Then, given the *LogicGraph* along with the annotations, Hetu’s graph compiler generates multiple *ExecGraphs*, one for each strategy. In essence, for each *ExecGraph*, based on the input tensors’ annotations and the operators that are applied to them, the graph compiler infers the sharding of output tensors, and inserts communication operators if needed. It then orchestrates the *ExecGraphs* to minimize hot switching cost. Finally, the runtime of Hetu executes the training routine with multiple *ExecGraphs*, which involves the execution of each individual graph, the model hot switching between two graphs, and the model update in the instantiation *ExecGraph*.

²Source code is available at: <https://github.com/PKU-DAIR/Hetu>. Note that it is also possible to implement parallelism hot switching via other frameworks like PyTorch. For example, we can turn a PyTorch model into an intermediate graph representation (IR) to facilitate the compilation.

Table 2. Models for evaluation.

Model	#Layers	#Heads	Hidden Dim
LLaMA2-7B	32	32	4096
LLaMA2-13B	40	40	5120
LLaMA2-32B	60	64	6656

We implement communication primitives by NCCL, and leverage libraries like FlashAttention [12, 13], cuBLAS, and cutlass for efficient computation. We follow Megatron-LM to incorporate DP (ZeRO-1), TP, PP, and Megatron-style SP. In the supplementary material, we demonstrate that when training with static parallelism strategies, our system has similar performance against Megatron-LM. We also apply the following optimizations for hot switching training.

Communication Group Management. Since different model hot switching requires different communication patterns, there would be a large number of NCCL communication groups. In order to minimize the expense of communication group construction and their memory consumption, we maintain a global pool of communication groups and try to reuse them throughout the training task if possible. In addition, communication groups for all parallelism strategies are also maintained and re-used in a similar vein.

Message Fusion. During model hot switching, if a source device needs to send multiple model slices to the same target device, we pack them together and trigger only one communication call to reduce the kernel launch overhead. Note that although this necessitates extra memory copy operations, they can be done via sequential memory accesses as long as the layout optimization technique is applied. Furthermore, we fuse the many-to-many communication via the batched NCCL send/recv primitives, which enables the concurrent transmission of model slices.

5 Experiments

5.1 Experimental Setup

Environments. Our experiments are conducted on 4 GPU servers, where each server is equipped with 8 NVIDIA A800-80GB GPUs. Within each server, the GPUs are connected via NVLink with a bandwidth of 400GB/s, and the servers are connected via Infini-band with a bandwidth of 200GB/s.

Baselines. We compare HotSPa with two LLM training systems, Megatron-LM and DeepSpeed, due to their widespread adoption. Megatron-LM encompasses DP (ZeRO-1), PP, TP, and Megatron-style SP to achieve hybrid parallelism training, whilst DeepSpeed supports the integration of ZeRO with different stages and the Ulysses-style SP. To achieve a fair comparison, we tune the strategies for both baselines and report the best values in all experiments.

Models. Since most LLMs have similar model architectures (i.e., decoder-only Transformer layers with casual masks), we focus on the LLaMA2 series LLMs due to their popularity. As detailed in Table 2, we consider three model sizes. For

both the 7B and 13B models, we conduct experiments with 8 and 16 GPUs, whilst for the 32B model, we use 32 GPUs.

Datasets. As introduced in Section 3, skewness is common in human texts, so the idea of parallelism hot switching holds for a wide range of datasets. We consider two datasets in our experiments, i.e., *CommonCrawl* and *GitHub*, and the distributions of sequence lengths are shown in Figure 3.

Protocols. For each pair of model and dataset, we vary the context length (s) from 4K to 32K to assess the performance of our work more comprehensively. We use the AdamW optimizer, and the global batch size of each training step is fixed as 512 by default. For HotSPa, we consider dividing sequences into 3 or 4 groups in most experiments. All results are averaged over 100 iterations.

5.2 End-to-end Evaluation

We first assess the training efficiency of by measuring the running time of each mini-batch of the three systems. The results are listed in Figure 7.

To begin with, according to the results of $s = 4K$, when training with static parallelism strategies, HotSPa has a similar performance compared with Megatron-LM, verifying the correctness of our system implementation. (Later in Section 5.3 we will show that we can further divide sequences shorter than 4K into more groups for better performance.)

However, as the context length increases, Megatron-LM needs to conform to more memory-saving but less efficient strategies in order to avoid out-of-memory error, leading to significant performance degradation. For instance, when training with the *CommonCrawl* dataset, where the proportion of sequences longer than 4K is lower than 1.5%, extending the model context length from 4K to 32K results in 2-4 $\times$ of increment in terms of running time of Megatron-LM. In contrast, by treating sequences with different levels of lengths separately, HotSPa is able to adopt efficient strategies to process the (relatively) short sequences, and gradually switch to more memory-saving strategies for longer sequences. As a result, the running time of HotSPa is not prolonged significantly w.r.t. the context length on the highly skewed dataset. Eventually, HotSPa outperforms Megatron-LM by up to 2.99 $\times$ on the *CommonCrawl* dataset.

Compared to *CommonCrawl*, the *GitHub* dataset contains much more long sequences, so the processing of long sequences accounts for a larger proportion of the end-to-end running time. Therefore, HotSPa has a smaller improvement on the *GitHub* dataset. Nevertheless, it still achieves up to 1.5 $\times$ of speedup, which is significant in the field of LLM training. Consequently, HotSPa also supports datasets with more long sequences well.

Note that although DeepSpeed performs well for LLaMA2-7B with context length lower than 16K, but has the worst performance in most other cases. Such performance differences are due to the use of ZeRO-3. When training with

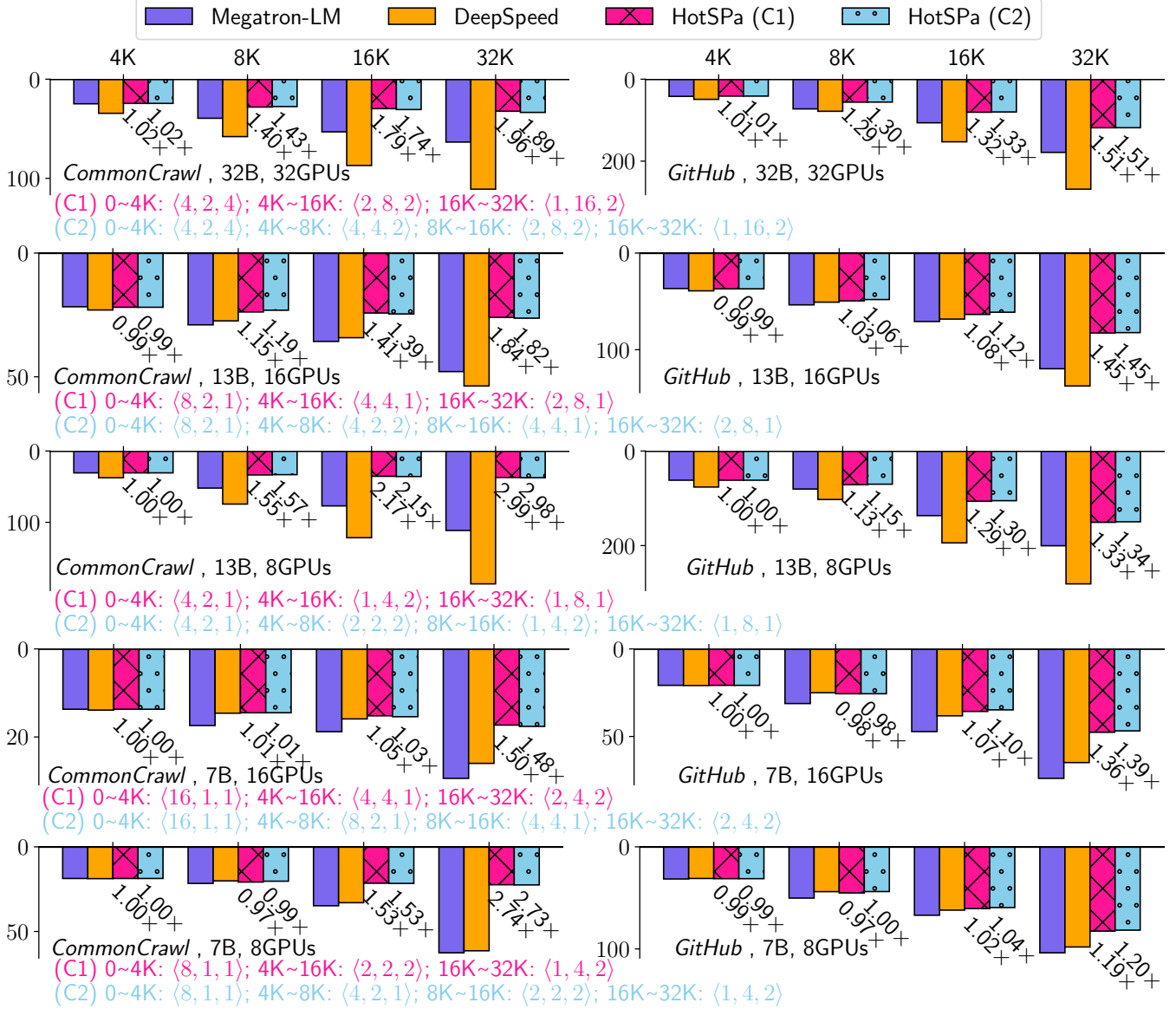


Figure 7. End-to-end evaluation (measured in seconds per mini-batch). We present two hot switching combinations for each experiments and the corresponding speedups compared with baselines (format: "seq_len_range: (DP, TP, PP)").

Table 3. Case studies of training with different combinations of parallelism strategies (32B, 32GPUs, s=32K). We dissect the running time for different groups of sequence lengths. "Others" includes the time cost of hot switching in one mini-batch.

Breakdown	CommonCrawl (time in seconds)				GitHub (time in seconds)			
	Static	C1	C2	C3	Static	C1	C2	C3
0~1K	42.1	5.4 (1.18×)	2.7 (1.29×)	12.8 (3.28×)	34.9	37.4 (1.40×)	15.4 (1.62×)	8.6 (4.05×)
1K~4K	12.5			8.2 (1.52×)	38.4			17.1 (2.24×)
4K~8K	3.5			2.7 (1.29×)	25.0			15.4 (1.62×)
8K~16K	2.9			2.2 (1.31×)	27.3			20.2 (1.35×)
16K~32K	2.4	2.4 (1.00×)	2.4 (1.00×)	2.4 (1.00×)	53.0	53.0 (1.00×)	53.0 (1.00×)	53.0 (1.00×)
Others	-	1.8	3.5	4.9	-	1.8	3.5	4.9
Total	63.4	32.3 (1.96×)	33.5 (1.89×)	33.2 (1.90×)	178.6	118.5 (1.50×)	118.4 (1.50×)	119.2 (1.49×)

(Static) 0~32K: (1, 16, 2) (C1) 0~4K: (4, 2, 4); 4K~16K: (2, 8, 2); 16K~32K: (1, 16, 2)

(C2) 0~4K: (4, 2, 4); 4K~8K: (4, 4, 2); 8K~16K: (2, 8, 2); 16K~32K: (1, 16, 2)

(C3) 0~1K: (8, 4, 1); 1K~4K: (4, 2, 4); 4K~8K: (4, 4, 2); 8K~16K: (2, 8, 2); 16K~32K: (1, 16, 2)

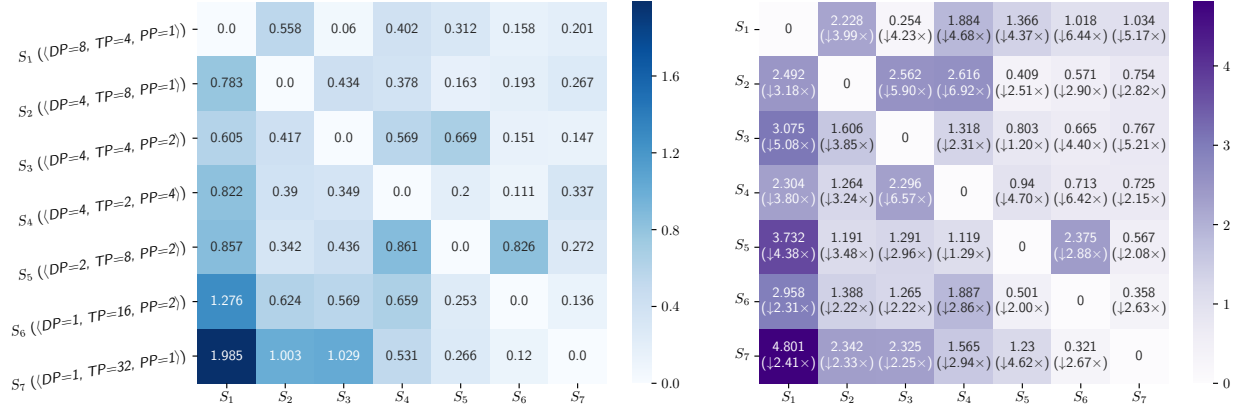


Figure 8. Time cost (in seconds) of switching between parallelism strategies (LLaMA2-32B, 32GPUs). The value in the i -th row and j -th column indicates switching from S_i to S_j . (Left: time cost of our work. Right: time cost without the optimizations in Section 4.3.)

relatively large models and/or higher context lengths, ZeRO-3 is necessary for DeepSpeed to accommodate the memory requirement. As introduced in Section 2.3, ZeRO-3 needs extra all-gather communication for each gradient accumulation step, resulting in excessive efficiency degradation. As a result, HotSPa provides better support for a wider range of LLM training scenarios, and outperforms DeepSpeed by up to 2.6 $\times$ and 5 $\times$ on the two datasets, respectively.

5.3 Case Studies

To anatomize the superior performance of HotSPa more in-depth, we study the running time of a few cases in detail. Specifically, we consider the training of LLaMA2-32B with a context length of 32K, and break down the running time of different combinations of parallelism strategies.

Table 3 shows the detailed running time for each group as well as the hot switching communications. It can be found that the majority of performance gains come from the processing of short sequences. For instance, when we use a static strategy, since it must be memory-saving enough to accommodate the training of long sequences, it takes 42.1 seconds to process the first group of sequences (i.e., sequences with lengths ranging from 0 to 1k) on the *CommonCrawl* dataset. However, by enabling parallelism hot switching, we can adopt a much more efficient strategy to handle such short sequences, achieving 3.28 $\times$ of speedup. Meanwhile, as shown in the left side of Figure 8, the overhead brought by model hot switching is minor. For instance, switching the model parameters from $\langle DP=1, TP=16, PP=2 \rangle$ to $\langle DP=8, TP=4, PP=1 \rangle$ takes less than 2 seconds, which is marginal compared with the aforementioned improvement. As a consequence, by separating the strategies, HotSPa accelerates the processing of all but the last groups of sequences, enhancing the overall training efficiency by a large extent.

Last but not least, we observe that the performance of HotSPa does not vary significantly w.r.t. the number of groups

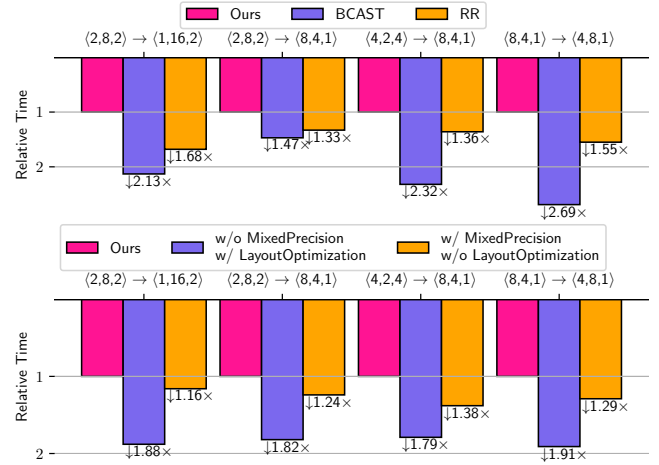


Figure 9. Ablation studies of hot switching.

that sequences are divided into. Intuitively speaking, having more groups enables a more fine-grained selection of parallelism strategies for different levels of sequence length, but increases the overhead of model hot switching. In all experiments, we do not meticulously design the sequence grouping and combination of strategies, whilst still achieving substantial improvements compared with state-of-the-art training systems. We believe this is essential for the system deployment in practical scenarios of LLM training.

5.4 Ablation Studies of Hot Switching

Section 5.3 has shown that the hot switching cost is quite small. This is attributed to the heuristic planning algorithm and the optimization techniques proposed in HotSPa. In this section, we evaluate the effectiveness of these techniques. Due to the space constraint, we only take several time-consuming cases of LLaMA2-32B as the representatives.

Heuristic Planning Algorithm. As discussed in Section 4.3, our planning algorithm is designed to prefer intra-node communication and minimize the maximum sending volume. To

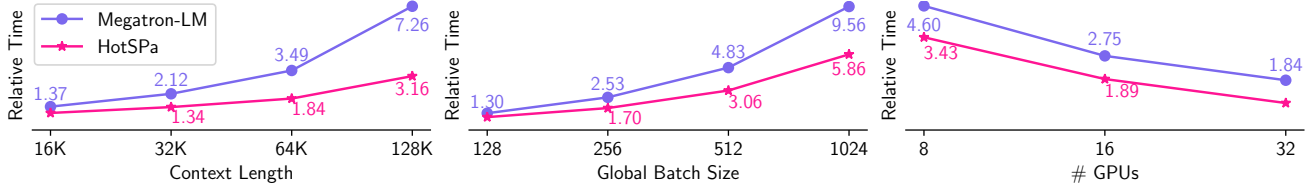


Figure 10. Sensitivity and scalability studies.

evaluate the effectiveness, we further implement two baselines: (1) BCAST: a naïve algorithm that ignores workload balance when the source strategy contains DP, and simply lets the device with the lowest rank broadcast the model; (2) RR: a round-robin algorithm that aims to balance the sending volume as much as possible but ignores the differences between intra- and inter-node bandwidth.

As presented in Figure 9, our heuristic algorithm outperforms the baselines in all cases, improving by 1.33-1.68 $\times$ and 1.47-2.69 $\times$, respectively. The empirical results verify that our heuristic algorithm is able to adapt to the key characteristics of modern networks between GPUs, and thus deduces efficient communication plans for hot switching.

Other Optimization techniques. In addition to the heuristic algorithm, HotSPa also adopts two techniques to enhance the hot switching efficiency. Firstly, HotSPa takes the mixed precision training into account and communicates only the 16-bit values. As shown in Figure 9, this significantly saves 79%-90% of hot switching cost. Secondly, the layout optimization technique, which is designed for column-partitioning TP, contributes 1.16-1.38 $\times$ of speedup. In short, both techniques suit the hot switching of LLMs well.

Last but not least, the effectiveness of the heuristic planning algorithm and the two optimization techniques are cumulative. As shown in the right side of Figure 8, compared to BCAST with the two optimization techniques disabled, our method achieves 1.2-6.57 times of improvement.

5.5 Sensitivity and Scalability

Finally, we compare the sensitivity and scalability of training with and without parallelism hot switching, represented by HotSPa and Megatron-LM. Due to the memory limitation, we consider the 7B model on *GitHub* over 16 GPUs. And unless specified, we set the context length as 32K and the global batch size as 512. The results are given in Figure 10.

Sensitivity w.r.t. Context Length. We first extend the context length to 64K and 128K to evaluate the performance of HotSPa. Empirical results prove that HotSPa achieves higher improvements compared with Megatron-LM for longer context lengths. This is reasonable since short sequences are always processed with efficient strategies regardless of the context length in HotSPa, whilst Megatron-LM conforms to a less efficient strategy when the context length increases.

Sensitivity w.r.t. Global Batch Size. The performance gains of our work mainly lie in workload imbalance across

sequences within the same mini-batch, so we conduct experiments with different global batch sizes. We observe that the running time of Megatron-LM grows almost proportionally w.r.t. the global batch sizes, whilst that of HotSPa does not. The reason is that when using a larger batch size, we will have more samples in the groups of high sequence lengths, which reduces the bubbles in PP. Consequently, HotSPa outperforms Megatron-LM by a greater extent when using a larger batch size, which is commonly seen in practical LLM training scenarios as introduced in Section 2.

Scalability w.r.t. #GPUs. To assess the scalability of our work, we train the 13B model with 8, 16, and 32 GPUs, respectively. As shown in Figure 10, both systems exhibit sub-linear scalability due to the communication overhead in distributed training. However, HotSPa consistently runs faster, validating the effectiveness of parallelism hot switching.

6 Related Work

Intermediate Representations (IRs) for Parallelization.

It is a common approach to represent deep learning models via graph-based IRs [1, 32, 33, 62, 71]. To optimize distributed training, a series of studies have proposed tensor annotation techniques that helps finding more efficient tensor partitioning and device mapping. HotSPa aims to annotating tensors with multiple strategies simultaneously so that we can generate and orchestrate numerous distributed computation graphs together based on the same logical computation graph. Therefore, HotSPa’s designs are orthogonal and can be combined with existing techniques in distributed IRs.

Optimization of Parallelism Strategies. Due to the increasing scale in models, many approaches have been developed to optimize parallelism strategies in distributed training. Many prior works focus on how to optimize a specific parallelism strategy, such as reducing the bubbles in PP [54] and accelerating ZeRO-based DP with communication reduction [72]. Another line of work studies how to automatically finds the most suitable hybrid parallelism strategy for a given training task, and a series of notable frameworks have been developed [6, 23, 44, 66, 74, 81]. Our work is orthogonal to these works as we target at the orchestration and hot switching of multiple strategies. Furthermore, the optimization techniques proposed in these work can be integrated with HotSPa such as optimizing the strategy used for each group.

Long Sequence Training. Due to the demand of longer context lengths, many efforts have been made to support long sequence training. In particular, several kinds of sequence parallelism strategies are meticulously designed for LLMs [4, 35, 40]. However, these strategies are not efficient in handling short sequences due to their severe communication overhead. HotSPa is fully compatible with these strategies. In addition, there are other works that support long sequence more efficiently from the algorithmic perspective [3, 27, 45, 77], which are orthogonal to our work.

Memory Optimization. In addition to the parallelization of models, there are many other techniques for memory reduction on a single device in large-scale model training, such as activation recomputation [10, 29], tensor offloading [58, 60, 61], and the combination of them [53, 73, 78]. These techniques typically reduces memory consumption at the price of extra recomputation or data transfer overheads. We can incorporate these techniques with HotSPa.

7 Limitations and Discussion

Despite the effectiveness demonstrated in Section 5, there are several limitations to our work.

Firstly, a major limitation is the reliance on user-provided combinations of strategies. To be more user-friendly, it is important to design approaches that automatically determine the optimal configuration of group partitioning and the corresponding parallelism strategy for each group, which necessitates solving a joint optimization problem.

Readers might suspect that we could decouple the problem, e.g., enumerating all plausible ways of group partitioning and utilizing automatic parallelism techniques to determine the optimal strategy for each group. However, this is still non-trivial to achieve. One challenge is that group partitioning influences the expected number of sequences falling into each group, which may change the optimal choice of strategy. Thus, it necessitates a more precise cost model to guide the automatic parallelism deduction. Another challenge is that the enumeration-based approach would lead to a high time cost, hampering its usability, so it would be important to incorporate pruning to avoid unnecessary enumeration. We would like to leave this as our future work.

Secondly, since the major overhead introduced by our technique is the time cost of hot switching, it may not work well when such overhead accounts for a large portion of the training time. For instance, currently, the number of gradient accumulation steps is large so that the hot switching overhead can be amortized. However, this may not be true when we scale the training task to more devices.

Ideally speaking, when there are more devices, a common approach is to increase the DP degree and meanwhile increase the global batch size proportionally. (For HotSPa, the number of strategies used is determined based on the context length, which is irrelevant to the number of devices. Thus,

we only need to increase the DP degrees of all used strategies and also increase the global batch size proportionally.) In this situation, the number of gradient accumulation steps remains the same level, which indicates that the hot switching overhead would not account for a larger portion. Thus, the parallelism hot switching technique is still effective.

Nevertheless, we cannot use an excessively huge global batch size since it would harm the convergence. Therefore, when the number of devices keeps increasing (e.g., several thousands or even more), the global batch size should remain stable rather than increasing proportionally. It indicates that the number of gradient accumulation steps decreases accordingly. Under such a circumstance, the hot switching overhead would become less worthy. Due to the high expense of GPUs, our experiments are conducted with up to 32 GPUs. However, it would be an important direction to empirically study when will the parallelism hot switching technique be (in)effective given a huge number of devices.

Thirdly, there are many parallelism strategies that are not incorporated into HotSPa yet, such as expert parallelism and context parallelism. It is essential to incorporate them in HotSPa to support a wider range of models and applications.

8 Conclusion

In this work, we pointed out the problem that leveraging a static parallelism strategy would lead to undesirable performance due to the skewness in sequence lengths. To address this problem, we proposed a novel distributed training paradigm of parallelism hot switching, which enables fine-grained selection of parallelism strategies for sequences with different levels of lengths. A brand new system, namely HotSPa, is developed in this paradigm for LLM training. HotSPa consists of a graph compilation process to generate and orchestrate the distributed computation graphs of multiple strategies simultaneously, as well as a heuristic planning algorithm that deduces efficient communication plans for the model transmission between any pair of strategies. Empirical results show that HotSPa can outperform existing systems by up to 2.99 $\times$ in terms of training speed.

Acknowledgments

We would like to thank our shepherd Paul Barham and the anonymous reviewers for their valuable feedback. This work is supported by National Science and Technology Major Project (2022ZD0116315), National Natural Science Foundation of China (U22B2037, U23B2048), Beijing Municipal Science and Technology Project (Z231100010323002), China National Postdoctoral Program for Innovative Talents (BX20230012), China Postdoctoral Science Foundation (2024M750103), Beijing Natural Science Foundation (4244080), research grant No. SH-2024JK29, PKU-Tencent joint research Lab, and High-performance Computing Platform of Peking University.

References

- [1] Martin Abadi, Paul Barham, Jianmin Chen, Zhifeng Chen, Andy Davis, Jeffrey Dean, Matthieu Devin, Sanjay Ghemawat, Geoffrey Irving, Michael Isard, Manjunath Kudlur, Josh Levenberg, Rajat Monga, Sherry Moore, Derek Gordon Murray, Benoit Steiner, Paul A. Tucker, Vijay Vasudevan, Pete Warden, Martin Wicke, Yuan Yu, and Xiaoqiang Zheng. 2016. TensorFlow: A System for Large-Scale Machine Learning. In *12th USENIX Symposium on Operating Systems Design and Implementation (OSDI 2016)*. 265–283.
- [2] Alexei Baevski, Yuhao Zhou, Abdelrahman Mohamed, and Michael Auli. 2020. wav2vec 2.0: A Framework for Self-Supervised Learning of Speech Representations. In *Annual Conference on Neural Information Processing Systems 2020 (NeurIPS 2020)*.
- [3] Iz Beltagy, Matthew E. Peters, and Arman Cohan. 2020. Longformer: The Long-Document Transformer. *CoRR* abs/2004.05150 (2020).
- [4] William Brandon, Aniruddha Nrusimha, Kevin Qian, Zachary Ankner, Tian Jin, Zhiye Song, and Jonathan Ragan-Kelley. 2023. Striped Attention: Faster Ring Attention for Causal Transformers. *CoRR* abs/2311.09431 (2023).
- [5] Tom B. Brown, Benjamin Mann, Nick Ryder, Melanie Subbiah, Jared Kaplan, Prafulla Dhariwal, Arvind Neelakantan, Pranav Shyam, Girish Sastry, Amanda Askell, Sandhini Agarwal, Ariel Herbert-Voss, Gretchen Krueger, Tom Henighan, Rewon Child, Aditya Ramesh, Daniel M. Ziegler, Jeffrey Wu, Clemens Winter, Christopher Hesse, Mark Chen, Eric Sigler, Matusz Litwin, Scott Gray, Benjamin Chess, Jack Clark, Christopher Berner, Sam McCandlish, Alec Radford, Ilya Sutskever, and Dario Amodei. 2020. Language Models are Few-Shot Learners. In *Annual Conference on Neural Information Processing Systems 2020 (NeurIPS 2020)*.
- [6] Zhenkun Cai, Xiao Yan, Kaihao Ma, Yidi Wu, Yuzhen Huang, James Cheng, Teng Su, and Fan Yu. 2022. TensorOpt: Exploring the Tradeoffs in Distributed DNN Training With Auto-Parallelism. *IEEE Transactions on Parallel and Distributed Systems (TPDS)* 33, 8 (2022), 1967–1981.
- [7] Kirk W. Cameron. 2023. Adventures Beyond Amdahl’s Law: How Power-Performance Measurement and Modeling at Scale Drive Server and Supercomputer Design. *J. Comput. Sci. Technol.* 38, 1 (2023), 80–86.
- [8] Hong Chen, Kang Yuan, Yanjun Huang, Lulu Guo, Yulei Wang, and Jie Chen. 2023. Feedback is all you need: from ChatGPT to autonomous driving. *Sci. China Inf. Sci.* 66, 6 (2023).
- [9] Tianqi Chen, Thierry Moreau, Ziheng Jiang, Lianmin Zheng, Eddie Q. Yan, Haichen Shen, Meghan Cowan, Leyuan Wang, Yuwei Hu, Luis Ceze, Carlos Guestrin, and Arvind Krishnamurthy. 2018. TVM: An Automated End-to-End Optimizing Compiler for Deep Learning. In *13th USENIX Symposium on Operating Systems Design and Implementation (OSDI 2018)*. 578–594.
- [10] Tianqi Chen, Bing Xu, Chiyuan Zhang, and Carlos Guestrin. 2016. Training Deep Nets with Sublinear Memory Cost. *CoRR* abs/1604.06174 (2016).
- [11] Aakanksha Chowdhery, Sharan Narang, Jacob Devlin, Maarten Bosma, Gaurav Mishra, Adam Roberts, Paul Barham, Hyung Won Chung, Charles Sutton, Sebastian Gehrmann, Parker Schuh, Kensen Shi, Sasha Tsvyashchenko, Joshua Maynez, Abhishek Rao, Parker Barnes, Yi Tay, Noam Shazeer, Vinodkumar Prabhakaran, Emily Reif, Nan Du, Ben Hutchinson, Reiner Pope, James Bradbury, Jacob Austin, Michael Isard, Guy Gur-Ari, Pengcheng Yin, Toju Duke, Anselm Levskaya, Sanjay Ghemawat, Sunipa Dev, Henryk Michalewski, Xavier Garcia, Vedant Misra, Kevin Robinson, Liam Fedus, Denny Zhou, Daphne Ippolito, David Luan, Hyeontaek Lim, Barret Zoph, Alexander Spiridonov, Ryan Sepassi, David Dohan, Shivani Agrawal, Mark Omernick, Andrew M. Dai, Thanumalayan Sankaranarayanan Pillai, Marie Pellat, Aitor Lewkowycz, Erica Moreira, Rewon Child, Oleksandr Polozov, Katherine Lee, Zongwei Zhou, Xuezhi Wang, Brennan Saeta, Mark Diaz, Orhan Firat, Michele Catasta, Jason Wei, Kathy Meier-Hellstern, Douglas Eck, Jeff Dean, Slav Petrov, and Noah Fiedel. 2023. PaLM: Scaling Language Modeling with Pathways. *Journal of Machine Learning Research (JMLR)* 24 (2023), 240:1–240:113.
- [12] Tri Dao. 2024. FlashAttention-2: Faster Attention with Better Parallelism and Work Partitioning. In *International Conference on Learning Representations 2024 (ICLR 2024)*.
- [13] Tri Dao, Daniel Y. Fu, Stefano Ermon, Atri Rudra, and Christopher Ré. 2022. FlashAttention: Fast and Memory-Efficient Exact Attention with IO-Awareness. In *Annual Conference on Neural Information Processing Systems 2022 (NeurIPS 2022)*.
- [14] Harm de Vries. 2023. In the long (context) run. "https://www.harmdevries.com/post/context-length/".
- [15] Jeffrey Dean, Greg Corrado, Rajat Monga, Kai Chen, Matthieu Devin, Quoc V. Le, Mark Z. Mao, Marc’Aurelio Ranzato, Andrew W. Senior, Paul A. Tucker, Ke Yang, and Andrew Y. Ng. 2012. Large Scale Distributed Deep Networks. In *26th Annual Conference on Neural Information Processing Systems 2012 (NeurIPS 2022)*. 1232–1240.
- [16] Jacob Devlin, Ming-Wei Chang, Kenton Lee, and Kristina Toutanova. 2018. BERT: Pre-training of Deep Bidirectional Transformers for Language Understanding. *CoRR* abs/1810.04805 (2018).
- [17] Alexey Dosovitskiy, Lucas Beyer, Alexander Kolesnikov, Dirk Weissenborn, Xiaohua Zhai, Thomas Unterthiner, Mostafa Dehghani, Matthias Minderer, Georg Heigold, Sylvain Gelly, Jakob Uszkoreit, and Neil Houlsby. 2021. An Image is Worth 16x16 Words: Transformers for Image Recognition at Scale. In *International Conference on Learning Representations 2021 (ICLR 2021)*.
- [18] Fangcheng Fu, Yuzheng Hu, Yihan He, Jiawei Jiang, Yingxia Shao, Ce Zhang, and Bin Cui. 2020. Don’t Waste Your Bits! Squeeze Activations and Gradients for Deep Neural Networks via TinyScript. In *International Conference on Machine Learning 2020 (ICML 2020)*. 3304–3314.
- [19] Jordan Hoffmann, Sebastian Borgeaud, Arthur Mensch, Elena Buchatskaya, Trevor Cai, Eliza Rutherford, Diego de Las Casas, Lisa Anne Hendricks, Johannes Welbl, Aidan Clark, Tom Hennigan, Eric Noland, Katie Millican, George van den Driessche, Bogdan Damoc, Aurelia Guy, Simon Osindero, Karen Simonyan, Erich Elsen, Jack W. Rae, Oriol Vinyals, and Laurent Sifre. 2022. Training Compute-Optimal Large Language Models. *CoRR* abs/2203.15556 (2022).
- [20] Yanping Huang, Youlong Cheng, Ankur Bapna, Orhan Firat, Dehao Chen, Mia Xu Chen, Hyoukjoong Lee, Jiquan Ngiam, Quoc V. Le, Yonghui Wu, and Zhifeng Chen. 2019. GPipe: Efficient Training of Giant Neural Networks using Pipeline Parallelism. In *Annual Conference on Neural Information Processing Systems 2019 (NeurIPS 2019)*. 103–112.
- [21] Sam Ade Jacobs, Masahiro Tanaka, Chengming Zhang, Minjia Zhang, Shuaiwen Leon Song, Samyam Rajbhandari, and Yuxiong He. 2023. DeepSpeed Ulysses: System Optimizations for Enabling Training of Extreme Long Sequence Transformer Models. *CoRR* abs/2309.14509 (2023).
- [22] Zhihao Jia, Oded Padon, James Thomas, Todd Warszawski, Matei Zaharia, and Alex Aiken. 2019. TASO: optimizing deep learning computation with automatic generation of graph substitutions. In *Proceedings of the 27th ACM Symposium on Operating Systems Principles (SOSP 2019)*. 47–62.
- [23] Zhihao Jia, Matei Zaharia, and Alex Aiken. 2019. Beyond Data and Model Parallelism for Deep Neural Networks. In *Proceedings of Machine Learning and Systems 2019 (MLSys 2019)*.
- [24] Jiawei Jiang, Fangcheng Fu, Tong Yang, and Bin Cui. 2018. SketchML: Accelerating Distributed Machine Learning with Data Sketches. In *Proceedings of the 2018 International Conference on Management of Data (SIGMOD 2018)*. 1269–1284.
- [25] Youhe Jiang, Fangcheng Fu, Xupeng Miao, Xiaonan Nie, and Bin Cui. 2023. OSDP: Optimal Sharded Data Parallel for Distributed Deep Learning. In *Proceedings of the Thirty-Second International Joint Conference on Artificial Intelligence (IJCAI 2023)*. 2142–2150.

- [26] Jared Kaplan, Sam McCandlish, Tom Henighan, Tom B. Brown, Benjamin Chess, Rewon Child, Scott Gray, Alec Radford, Jeffrey Wu, and Dario Amodei. 2020. Scaling Laws for Neural Language Models. *CoRR* abs/2001.08361 (2020).
- [27] Angelos Katharopoulos, Apoorv Vyas, Nikolaos Pappas, and François Fleuret. 2020. Transformers are RNNs: Fast Autoregressive Transformers with Linear Attention. In *International Conference on Machine Learning 2020 (ICML 2020)*. 5156–5165.
- [28] Diederik P. Kingma and Jimmy Ba. 2015. Adam: A Method for Stochastic Optimization. In *3rd International Conference on Learning Representations 2015 (ICLR 2015)*.
- [29] Marisa Kirisame, Steven Lyubomirsky, Altan Haan, Jennifer Brennan, Mike He, Jared Roesch, Tianqi Chen, and Zachary Tatlock. 2021. Dynamic Tensor Rematerialization. In *International Conference on Learning Representations 2021 (ICLR 2021)*.
- [30] Vijay Korthikanti, Jared Casper, Sangkug Lym, Lawrence McAfee, Michael Andersch, Mohammad Shoeybi, and Bryan Catanzaro. 2022. Reducing Activation Recomputation in Large Transformer Models. *CoRR* abs/2205.05198 (2022).
- [31] Mario Michael Krell, Matej Kosec, Sergio P Perez, and Andrew Fitzgibbon. 2021. Efficient sequence packing without cross-contamination: Accelerating large language models without impacting performance. *CoRR* abs/2107.02027 (2021).
- [32] Chris Lattner, Mehdi Amini, Uday Bondhugula, Albert Cohen, Andy Davis, Jacques A. Pienaar, River Riddle, Tatiana Shpeisman, Nicolas Vasilache, and Aleksandr Zinenko. 2021. MLIR: Scaling Compiler Infrastructure for Domain Specific Computation. In *International Symposium on Code Generation and Optimization 2021 (CGO 2021)*. 2–14.
- [33] Tung D. Le, Gheorghe-Teodor Bercea, Tong Chen, Alexandre E. Eichenberger, Haruki Imai, Tian Jin, Kiyokuni Kawachiya, Yasushi Negishi, and Kevin O'Brien. 2020. Compiling ONNX Neural Network Models Using MLIR. *CoRR* abs/2008.08272 (2020).
- [34] Zhi Lei, Guixian Zhang, Lijuan Wu, Kui Zhang, and Rongjiao Liang. 2022. A Multi-level Mesh Mutual Attention Model for Visual Question Answering. *Data Sci. Eng.* 7, 4 (2022), 339–353.
- [35] Dacheng Li, Rulin Shao, Anze Xie, Eric P. Xing, Joseph E. Gonzalez, Ion Stoica, Xuezhe Ma, and Hao Zhang. 2023. LightSeq: Sequence Level Parallelism for Distributed Training of Long Context Transformers. *CoRR* abs/2310.03294 (2023).
- [36] Mingzhen Li, Yi Liu, Xiaoyan Liu, Qingxiao Sun, Xin You, Hailong Yang, Zhongzhi Luan, Lin Gan, Guangwen Yang, and Depei Qian. 2021. The Deep Learning Compiler: A Comprehensive Survey. *IEEE Transactions on Parallel and Distributed Systems (TPDS)* 32, 3 (2021), 708–727.
- [37] Shenggui Li, Fuzhao Xue, Chaitanya Baranwal, Yongbin Li, and Yang You. [n. d.]. Sequence parallelism: Long sequence training from system perspective. *CoRR* abs/2105.13120 ([n. d.]).
- [38] Shen Li, Yanli Zhao, Rohan Varma, Omkar Salpekar, Pieter Noordhuis, Teng Li, Adam Paszke, Jeff Smith, Brian Vaughan, Pritam Damania, and Soumith Chintala. 2020. PyTorch Distributed: Experiences on Accelerating Data Parallel Training. *Proc. VLDB Endow.* 13, 12 (2020), 3005–3018.
- [39] Haotian Liu, Chunyuan Li, Qingyang Wu, and Yong Jae Lee. 2023. Visual Instruction Tuning. In *Annual Conference on Neural Information Processing Systems 2023 (NeurIPS 2023)*.
- [40] Hao Liu, Matei Zaharia, and Pieter Abbeel. 2023. Ring Attention with Blockwise Transformers for Near-Infinite Context. *CoRR* abs/2310.01889 (2023).
- [41] Ze Liu, Yutong Lin, Yue Cao, Han Hu, Yixuan Wei, Zheng Zhang, Stephen Lin, and Baining Guo. 2021. Swin Transformer: Hierarchical Vision Transformer using Shifted Windows. In *IEEE/CVF International Conference on Computer Vision 2021 (ICCV 2021)*. IEEE, 9992–10002.
- [42] Ilya Loshchilov and Frank Hutter. 2019. Decoupled Weight Decay Regularization. In *7th International Conference on Learning Representations 2019 (ICLR 2019)*.
- [43] Xupeng Miao, Xiaonan Nie, Hailin Zhang, Tong Zhao, and Bin Cui. 2023. Hetu: a highly efficient automatic parallel distributed deep learning system. *Sci. China Inf. Sci.* 66, 1 (2023).
- [44] Xupeng Miao, Yujie Wang, Youhe Jiang, Chunan Shi, Xiaonan Nie, Hailin Zhang, and Bin Cui. 2022. Galvatron: Efficient Transformer Training over Multiple GPUs Using Automatic Parallelism. *Proc. VLDB Endow.* 16, 3 (2022), 470–479.
- [45] Xupeng Miao, Shenhan Zhu, Fangcheng Fu, Ziyu Guo, Zhi Yang, Yaofeng Tu, Zhihao Jia, and Bin Cui. 2024. X-former Elucidator: Reviving Efficient Attention for Long Context Language Modeling. In *Proceedings of the Thirty-Fourth International Joint Conference on Artificial Intelligence (IJCAI 2024)*. 8179–8187.
- [46] Paulius Micikevicius, Sharan Narang, Jonah Alben, Gregory F. Diamos, Erich Elsen, David Garcia, Boris Ginsburg, Michael Houston, Oleksii Kuchaiev, Ganesh Venkatesh, and Hao Wu. 2018. Mixed Precision Training. In *6th International Conference on Learning Representations 2018 (ICLR 2018)*.
- [47] Deepak Narayanan, Aaron Harlap, Amar Phanishayee, Vivek Seshadri, Nikhil R. Devanur, Gregory R. Ganger, Phillip B. Gibbons, and Matei Zaharia. 2019. PipeDream: generalized pipeline parallelism for DNN training. In *Proceedings of the 27th ACM Symposium on Operating Systems Principles (SOSP 2019)*. 1–15.
- [48] Deepak Narayanan, Amar Phanishayee, Kaiyu Shi, Xie Chen, and Matei Zaharia. 2021. Memory-Efficient Pipeline-Parallel DNN Training. In *International Conference on Machine Learning 2021 (ICML 2021)*. 7937–7947.
- [49] Deepak Narayanan, Mohammad Shoeybi, Jared Casper, Patrick LeGresley, Mostofa Patwary, Vijay Korthikanti, Dmitri Vainbrand, Prithvi Kashinkunti, Julie Bernauer, Bryan Catanzaro, Amar Phanishayee, and Matei Zaharia. 2021. Efficient large-scale language model training on GPU clusters using megatron-LM. In *International Conference for High Performance Computing, Networking 2021 (SC 2021)*. 58.
- [50] Xiaonan Nie, Yi Liu, Fangcheng Fu, Jinbao Xue, Dian Jiao, Xupeng Miao, Yangyu Tao, and Bin Cui. 2023. Angel-PTM: A Scalable and Economical Large-scale Pre-training System in Tencent. *Proc. VLDB Endow.* 16, 12 (2023), 3781–3794.
- [51] OpenAI. 2022. ChatGPT: Optimizing Language Models for Dialogue. "https://openai.com/blog/chatgpt".
- [52] OpenAI. 2023. GPT-4 Technical Report. *CoRR* abs/2303.08774 (2023).
- [53] Xuan Peng, Xuanhua Shi, Hulin Dai, Hai Jin, Weiliang Ma, Qian Xiong, Fan Yang, and Xuehai Qian. 2020. Capuchin: Tensor-based GPU Memory Management for Deep Learning. In *Architectural Support for Programming Languages and Operating Systems 2020 (ASPLOS 2020)*. 891–905.
- [54] Penghui Qi, Xinyi Wan, Guangxing Huang, and Min Lin. 2024. Zero Bubble Pipeline Parallelism. *CoRR* abs/2401.10241 (2024).
- [55] Alec Radford, Jong Wook Kim, Tao Xu, Greg Brockman, Christine McLeavey, and Ilya Sutskever. 2023. Robust Speech Recognition via Large-Scale Weak Supervision. In *International Conference on Machine Learning (ICML 2023)*. 28492–28518.
- [56] Colin Raffel, Noam Shazeer, Adam Roberts, Katherine Lee, Sharan Narang, Michael Matena, Yanqi Zhou, Wei Li, and Peter J. Liu. 2020. Exploring the Limits of Transfer Learning with a Unified Text-to-Text Transformer. *Journal of Machine Learning Research (JMLR)* 21 (2020), 140:1–140:67.
- [57] Samyam Rajbhandari, Jeff Rasley, Olatunji Ruwase, and Yuxiong He. 2020. ZeRO: memory optimizations toward training trillion parameter models. In *Proceedings of the International Conference for High Performance Computing, Networking, Storage and Analysis (SC 2020)*. 20.
- [58] Samyam Rajbhandari, Olatunji Ruwase, Jeff Rasley, Shaden Smith, and Yuxiong He. 2021. ZeRO-infinity: breaking the GPU memory wall for extreme scale deep learning. In *International Conference for High*

- Performance Computing, Networking 2021 (SC 2021)*. 59.
- [59] Jeff Rasley, Samyam Rajbhandari, Olatunji Ruwase, and Yuxiong He. 2020. DeepSpeed: System Optimizations Enable Training Deep Learning Models with Over 100 Billion Parameters. In *The 26th ACM SIGKDD Conference on Knowledge Discovery and Data Mining (KDD 2020)*. 3505–3506.
 - [60] Jie Ren, Samyam Rajbhandari, Reza Yazdani Aminabadi, Olatunji Ruwase, Shuangyan Yang, Minjia Zhang, Dong Li, and Yuxiong He. 2021. ZeRO-Offload: Democratizing Billion-Scale Model Training. In *2021 USENIX Annual Technical Conference (ATC 2021)*. 551–564.
 - [61] Minsoo Rhu, Natalia Gimelshein, Jason Clemons, Arslan Zulfiqar, and Stephen W. Keckler. 2016. vDNN: Virtualized deep neural networks for scalable, memory-efficient neural network design. In *49th Annual IEEE/ACM International Symposium on Microarchitecture (MICRO 2016)*. 18:1–18:13.
 - [62] Keshav Santhanam, Siddharth Krishna, Ryota Tomioka, Tim Harris, and Matei Zaharia. 2021. DistIR: An Intermediate Representation and Simulator for Efficient Neural Network Distribution. *CoRR* abs/2111.05426 (2021).
 - [63] Alexander Sergeev and Mike Del Balso. 2018. Horovod: fast and easy distributed deep learning in TensorFlow. *CoRR* abs/1802.05799 (2018).
 - [64] Mohammad Shoeybi, Mostofa Patwary, Raul Puri, Patrick LeGresley, Jared Casper, and Bryan Catanzaro. 2019. Megatron-LM: Training Multi-Billion Parameter Language Models Using Model Parallelism. *CoRR* abs/1909.08053 (2019).
 - [65] Xian-He Sun and Xiaoyang Lu. 2023. The Memory-Bounded Speedup Model and Its Impacts in Computing. *J. Comput. Sci. Technol.* 38, 1 (2023), 64–79.
 - [66] Jakub Tarnawski, Deepak Narayanan, and Amar Phanishayee. 2021. Piper: Multidimensional Planner for DNN Parallelization. In *Annual Conference on Neural Information Processing Systems 2021 (NeurIPS 2021)*. 24829–24840.
 - [67] Hugo Touvron, Louis Martin, Kevin Stone, Peter Albert, Amjad Almahairi, Yasmine Babaei, Nikolay Bashlykov, Soumya Batra, Prajjwal Bhargava, Shruti Bhosale, Dan Bikel, Cristian Canton-Ferrer, Moya Chen, Guillem Cucurull, David Esiobu, Jude Fernandes, Jeremy Fu, Wenyin Fu, Brian Fuller, Cynthia Gao, Vedanuj Goswami, Naman Goyal, Anthony Hartshorn, Saghar Hosseini, Rui Hou, Hakan Inan, Marcin Kardas, Viktor Kerkez, Madian Khabsa, Isabel Kloumann, Artem Korenev, Punit Singh Koura, Marie-Anne Lachaux, Thibaut Lavril, Jenya Lee, Diana Liskovich, Yinghai Lu, Yuning Mao, Xavier Martinet, Todor Mihaylov, Pushkar Mishra, Igor Molybog, Yixin Nie, Andrew Poulton, Jeremy Reizenstein, Rashi Rungta, Kalyan Saladi, Alan Schelten, Ruan Silva, Eric Michael Smith, Ranjan Subramanian, Xiaoqing Ellen Tan, Binh Tang, Ross Taylor, Adina Williams, Jian Xi-ang Kuan, Puxin Xu, Zheng Yan, Iliyan Zarov, Yuchen Zhang, Angela Fan, Melanie Kambadur, Sharan Narang, Aurélien Rodriguez, Robert Stojnic, Sergey Edunov, and Thomas Scialom. 2023. Llama 2: Open Foundation and Fine-Tuned Chat Models. *CoRR* abs/2307.09288 (2023).
 - [68] Colin Unger, Zhihao Jia, Wei Wu, Sina Lin, Mandeep Baines, Carlos Efrain Quintero Narvaez, Vinay Ramakrishnaiah, Nirmal Prajapati, Patrick S. McCormick, Jamaludin Mohd-Yusof, Xi Luo, Dheevatsa Mudigere, Jongsoo Park, Misha Smelyanskiy, and Alex Aiken. 2022. Unity: Accelerating DNN Training Through Joint Optimization of Algebraic Transformations and Parallelization. In *16th USENIX Symposium on Operating Systems Design and Implementation (OSDI 2022)*. 267–284.
 - [69] Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N. Gomez, Lukasz Kaiser, and Illia Polosukhin. 2017. Attention is All you Need. In *Annual Conference on Neural Information Processing Systems 2017 (NeurIPS 2017)*. 5998–6008.
 - [70] Changhan Wang, Anne Wu, Juan Pino, Alexei Baevski, Michael Auli, and Alexis Conneau. 2021. Large-Scale Self- and Semi-Supervised Learning for Speech Translation. In *22nd Annual Conference of the International Speech Communication Association (Interspeech 2021)*. 2242–2246.
 - [71] Fei Wang, Guoyang Chen, Weifeng Zhang, and Tiark Rompf. 2019. Parallel Training via Computation Graph Transformation. In *IEEE International Conference on Big Data 2019 (BigData 2019)*. 3430–3439.
 - [72] Guanhua Wang, Heyang Qin, Sam Ade Jacobs, Connor Holmes, Samyam Rajbhandari, Olatunji Ruwase, Feng Yan, Lei Yang, and Yuxiong He. 2023. ZeRO++: Extremely Efficient Collective Communication for Giant Model Training. *CoRR* abs/2306.10209 (2023).
 - [73] Linnan Wang, Jinmian Ye, Yiyang Zhao, Wei Wu, Ang Li, Shuaiwen Leon Song, Zenglin Xu, and Tim Kraska. [n. d.]. Superneurons: dynamic GPU memory management for training deep neural networks. In *Proceedings of the 23rd ACM SIGPLAN Symposium on Principles and Practice of Parallel Programming (PPoPP 2018)*. 41–53.
 - [74] Yujie Wang, Youhe Jiang, Xupeng Miao, Fangcheng Fu, Xiaonan Nie, and Bin Cui. 2023. Improving Automatic Parallel Training via Balanced Memory Workload Optimization. *CoRR* abs/2307.02031 (2023).
 - [75] Shuo Xiao, Dongqing Zhu, Chaogang Tang, and Zhenzhen Huang. 2023. Combining Graph Contrastive Embedding and Multi-head Cross-Attention Transfer for Cross-Domain Recommendation. *Data Sci. Eng.* 8, 3 (2023), 247–262.
 - [76] Yuanzhong Xu, HyoukJoong Lee, Dehao Chen, Blake A. Hechtman, Yanping Huang, Rahul Joshi, Maxim Krikun, Dmitry Lepikhin, Andy Ly, Marcello Maggioni, Ruoming Pang, Noam Shazeer, Shibo Wang, Tao Wang, Yonghui Wu, and Zhifeng Chen. 2021. GSPMD: General and Scalable Parallelization for ML Computation Graphs. *CoRR* abs/2105.04663 (2021).
 - [77] Manzil Zaheer, Guru Guruganesh, Kumar Avinava Dubey, Joshua Ainslie, Chris Alberti, Santiago Ontañón, Philip Pham, Anirudh Ravula, Qifan Wang, Li Yang, and Amr Ahmed. 2020. Big Bird: Transformers for Longer Sequences. In *Annual Conference on Neural Information Processing Systems 2020 (NeurIPS 2020)*.
 - [78] Pinxue Zhao, Hailin Zhang, Fangcheng Fu, Xiaonan Nie, Qibin Liu, Fang Yang, Yuanbo Peng, Dian Jiao, Shuaipeng Li, Jinbao Xue, Yangyu Tao, and Bin Cui. 2024. Efficiently Training 7B LLM with 1 Million Sequence Length on 8 GPUs. *CoRR* abs/2407.12117 (2024).
 - [79] Yanli Zhao, Andrew Gu, Rohan Varma, Liang Luo, Chien-Chin Huang, Min Xu, Less Wright, Hamid Shojanazeri, Mylène Ott, Sam Shleifer, Alban Desmaison, Can Balioglu, Pritam Damania, Bernard Nguyen, Geeta Chauhan, Yuchen Hao, Ajit Mathews, and Shen Li. 2023. PyTorch FSDP: Experiences on Scaling Fully Sharded Data Parallel. *Proc. VLDB Endow.* 16, 12 (2023), 3848–3860.
 - [80] Lianmin Zheng, Chengfan Jia, Minmin Sun, Zhao Wu, Cody Hao Yu, Ameer Haj-Ali, Yida Wang, Jun Yang, Danyang Zhuo, Koushik Sen, Joseph E. Gonzalez, and Ion Stoica. 2020. Ansor: Generating High-Performance Tensor Programs for Deep Learning. In *14th USENIX Symposium on Operating Systems Design and Implementation (OSDI 2020)*. 863–879.
 - [81] Lianmin Zheng, Zhuohan Li, Hao Zhang, Yonghao Zhuang, Zhifeng Chen, Yanping Huang, Yida Wang, Yuanzhong Xu, Danyang Zhuo, Eric P. Xing, Joseph E. Gonzalez, and Ion Stoica. 2022. Alpa: Automating Inter- and Intra-Operator Parallelism for Distributed Deep Learning. In *16th USENIX Symposium on Operating Systems Design and Implementation (OSDI 2022)*. 559–578.
 - [82] Yonghao Zhuang, Lianmin Zheng, Zhuohan Li, Eric P. Xing, Qirong Ho, Joseph Gonzalez, Ion Stoica, Hao Zhang, and Hexu Zhao. 2023. On Optimizing the Communication of Model Parallelism. In *Proceedings of Machine Learning and Systems 2023 (MLSys 2023)*.